# MuJoCo & imitation learning

Simulate motion. Learn behavior from an expert.

## Part 1 · MuJoCo

Physics and experimentation
CPU and GPU roles

## Part 2 · Imitation

Expert demonstrations
Learning and drift

**22 min explanation · 18 min demo · 5 min recap**

# Part 1 · MuJoCo

Why this simulator fits robot-learning experiments.

## Control inputs → physics → motion

Experiment → Reset → Repeat

# What is MuJoCo?

A physics simulator for articulated bodies and control.




**MuJoCo computes motion. The controller chooses actions.**

# Why simulation for RL?

Learning through interaction needs many trials.

### Repeat

Run many experiments.

### Reset

Try new initial conditions.

### Compare

Share scenarios and seeds.

### Scale

Collect trials in parallel.

**Explore in simulation before committing to hardware.**

# MuJoCo versus Gazebo

Choose the tool around the experiment.

## MuJoCo

Control + learning loops
Direct state/action access
GPU simulation backends

Tradeoff: add ROS integration.

## Gazebo

ROS + sensor workflows
Worlds, systems and plugins
Hardware-like integration

Tradeoff: broader system setup.

**Both can support learning. Their workflows emphasize different needs.**

# GPU compatibility: three different jobs

Drawing, learning, and physics are separate workloads.

### Rendering

Draw images.

### Policy training

Fit model parameters.

### Physics

Advance robot states.

**Standard MuJoCo: CPU physics. MJX / Warp: GPU simulation backends.**

**This demo: CPU physics + NumPy fitting. No GPU training.**

# Part 2 · Imitation learning

Learn an observation-to-action policy from demonstrations.

## Expert examples → labeled data → policy

Demonstrate → Train → Evaluate

# From computing a policy to learning one

Keep the policy idea; change where it comes from.

## Week 6

Given a model, compute a policy using Bellman equations.

## Week 7

Given expert examples, fit an observation-to-action policy.

**A policy still answers: what action should I take now?**

# What is imitation learning?

Use expert behavior to learn how to act.

### Human

Demonstrates actions.

### Controller

Computes feedback.

### Planner

Provides decisions.

Different experts can supply the demonstrations.

**This week: behavior cloning from observation/action labels.**

# The behavior-cloning workflow

Separate expert collection from independent execution.




**Expert present during collection**

**Learner independent at deployment**

**During deployment: current observation → learned action → new observation.**

# Demonstrations are labeled data

Inputs: what the expert sees. Labels: what it does.

### Observation · input

[error_x, error_y, velocity_x, velocity_y]

### Expert action · label

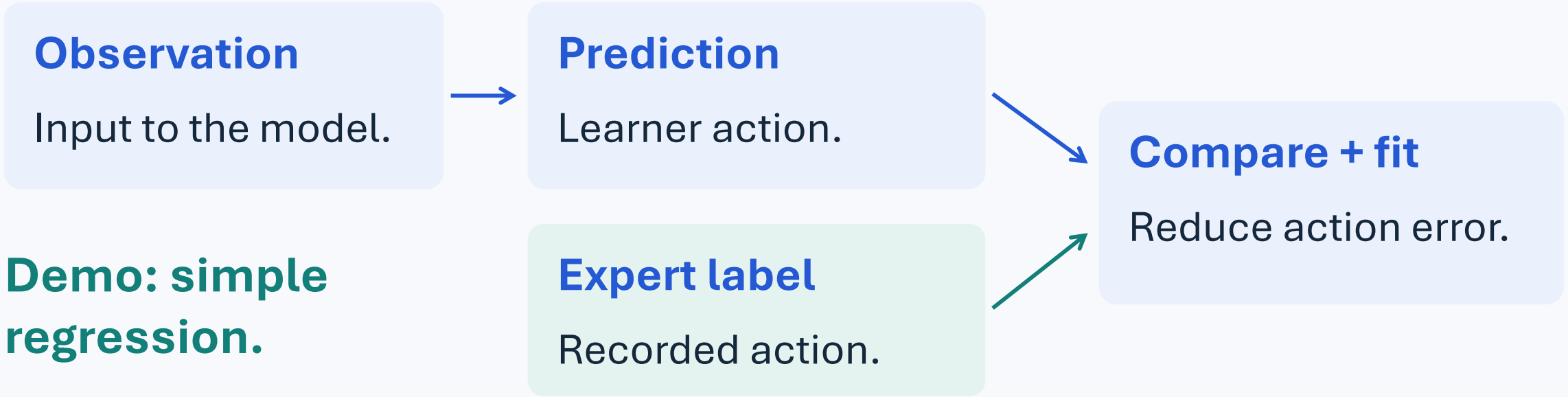
[force_x, force_y]

**Example: [0.3, −0.2, 0.1, 0] → [0.9, −0.8] N**

**100 episodes × 250 decisions = 25,000 labeled pairs.**

# Train it like supervised learning

Predict expert actions and reduce label error.




**Demo: simple regression.**

**No reward optimization in behavior-cloning training.**

# When imitation drifts

Our actions change the next inputs we will see.

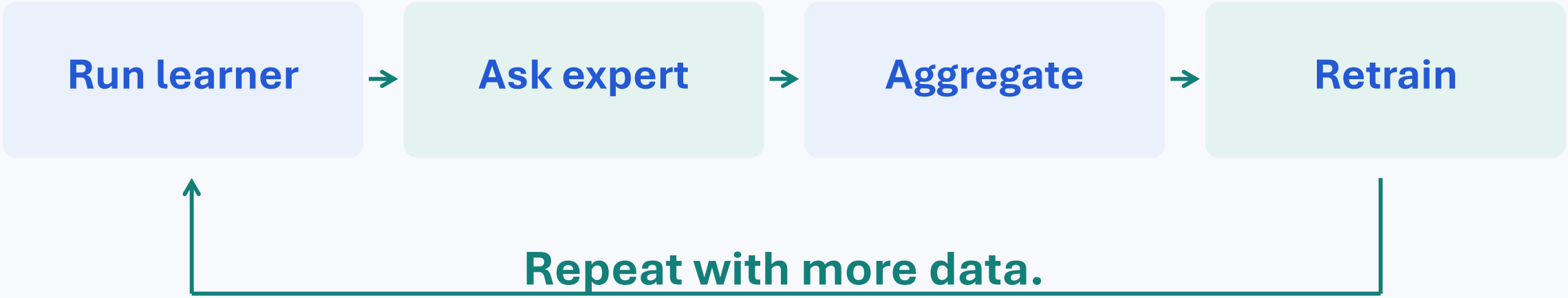
**Small error** → **Changed state** → **Unfamiliar input** → **Further error**




**Low action error on expert data does not guarantee successful control.**

# DAgger: learn from learner-visited states

Ask the expert about the states your learner actually reaches.




**Aggregate the new labels with earlier data, then retrain.**

# Other approaches: IRL and GAIL

## IRL

Infer a reward that explains expert behavior; use it to learn or plan a policy.

## GAIL

Train a policy to match expert state/action visitation using discriminator feedback.

**Different approaches; our demo uses behavior cloning.**

# Evaluate predictions and behavior

Two different questions need two different measurements.

## Held-out action error

Do predictions match expert labels?

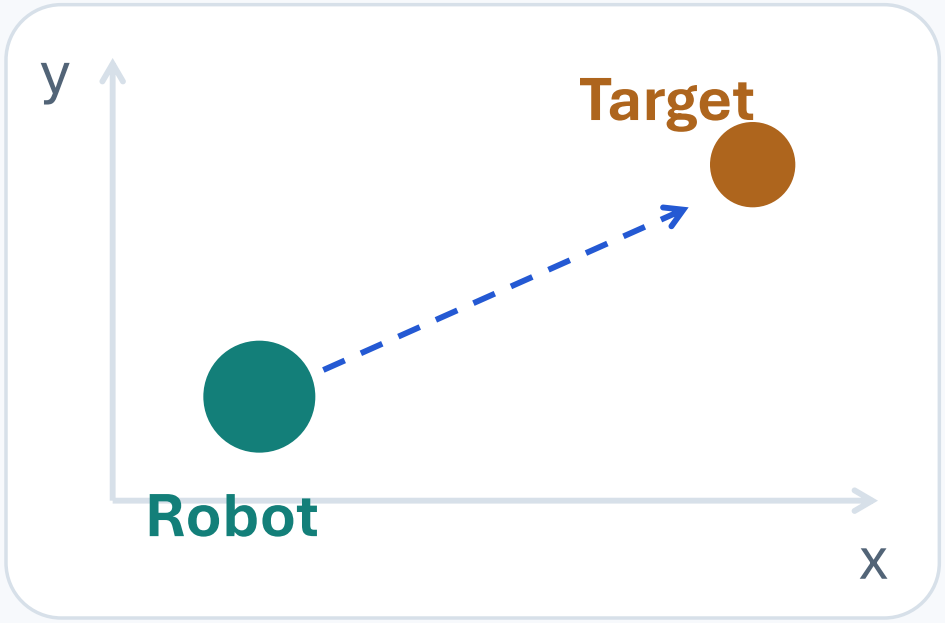
## Closed-loop success

Does the learner complete the task on fresh episodes?

**Split by whole trajectories. Compare on shared unseen cases.**

# Our coming demo: reach the target

A two-axis moving body, not an arm or grasping task.




## Observation

Position error + velocity

## Action

Two forces, each limited to ±10 N.

**Observe error + velocity. Command x/y forces.**

# Our expert is a feedback controller

A transparent teacher for an intentionally simple task.

## Position feedback

Pull toward the target.

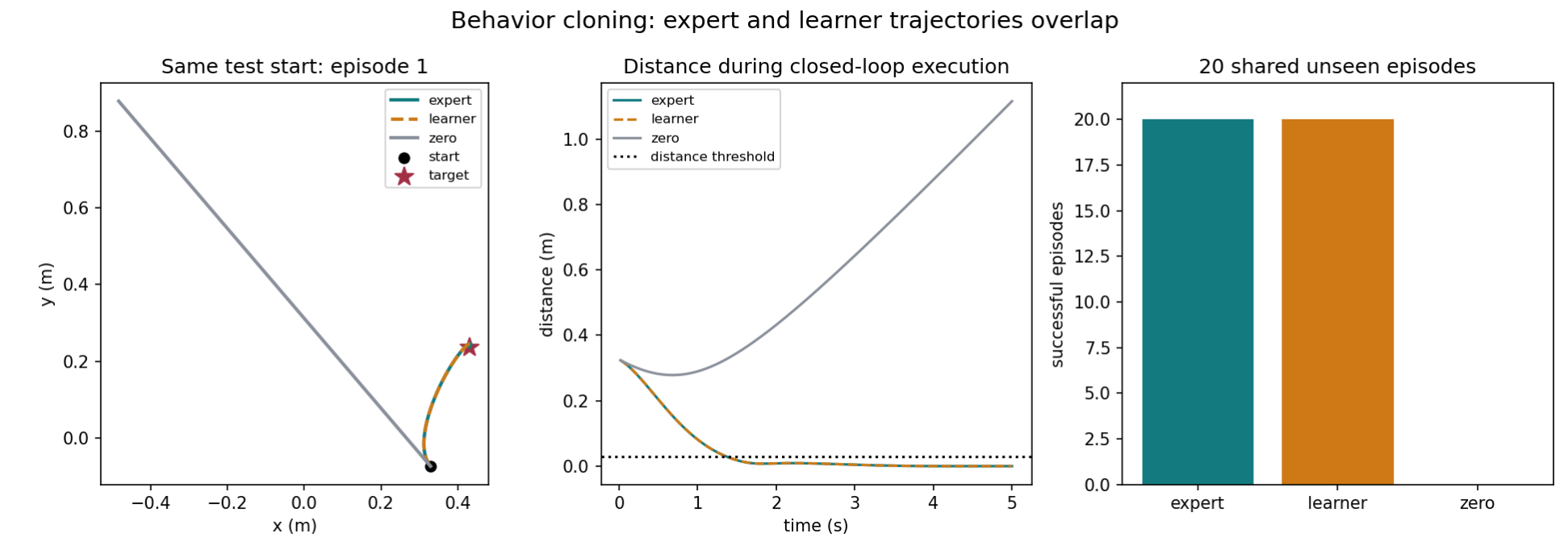
## Velocity feedback

Brake the motion to settle.

**The learner sees examples; it is not given the controller gains.**

# What should we expect?

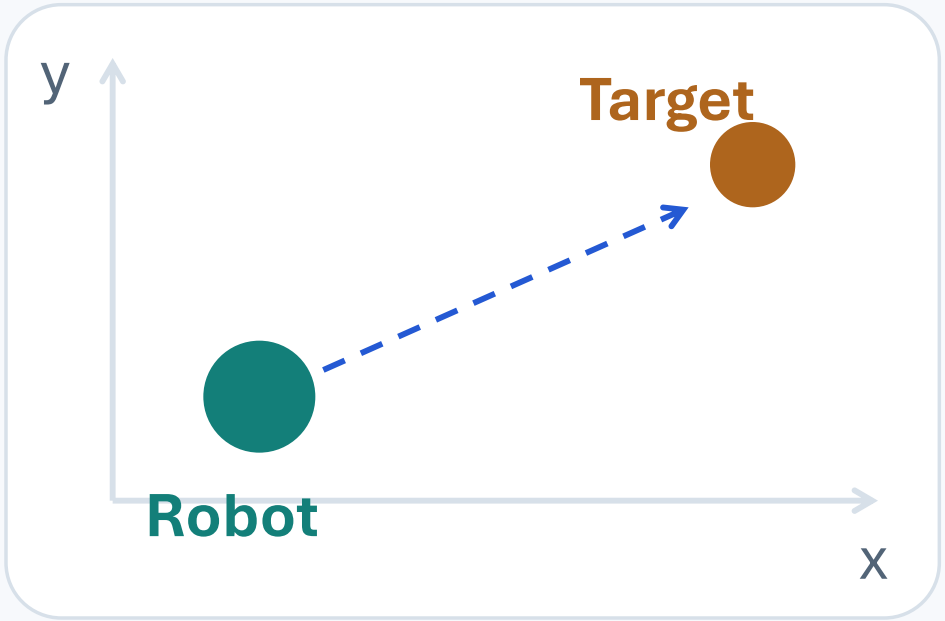
Expert and learner trajectories nearly overlap.




**Saved comparison · simple task, not a robustness guarantee.**

# Watch → collect → learn → compare

First meet the expert, then let the learner act independently.




**Watch the expert.**

**Collect and fit labels.**

**Watch the independent learner.**

**4 min expert · 4 collect · 3 train · 5 compare · 2 metrics**

# Questions, recap & HW4

Who provides dynamics, labels, and actions?

### Simulation

Why use MuJoCo for learning experiments?

### Imitation

How do expert actions become labels?

### Drift

Why test complete rollouts?

**MuJoCo simulates. Expert labels. Learner acts.**

OPTIONAL / DEEPER DISCUSSION

# CPU and GPU: choose the workload

Optional details, without setup instructions.

## Standard MuJoCo

CPU physics stepping
OpenGL visualization
Fits this small demo

## Accelerator backends

MJX / MuJoCo Warp
Many parallel environments
Check workload and support

**Rendering ≠ policy training ≠ physics stepping.**

OPTIONAL / DEEPER DISCUSSION

# Controller and learned weights

Supplemental detail for the demo, not a regression derivation.

$$\text{Force} = \text{clip}(4 \times \text{error} - 3 \times \text{velocity}, -10, 10)$$

$$\begin{bmatrix} 4 & 0 & -3 & 0 \\ 0 & 4 & 0 & -3 \end{bmatrix}$$

**W: 2 × 4**
**b ≈ [0, 0]**

Example: [0.3, −0.2, 0.1, 0] → [0.9, −0.8] N

**Coefficients are estimated from labels, not copied into the learner.**

OPTIONAL / DEEPER DISCUSSION

# Four approaches, different feedback

Optional comparison of the learning signal.

### Behavior cloning

Expert action labels.

### DAgger

Labels at learner-visited states.

### IRL

Infer reward explanations.

### GAIL

Discriminator feedback.

OPTIONAL / DEEPER DISCUSSION

# Data and evaluation checklist

Avoid mistaking prediction accuracy for control performance.

## Dataset

100 episodes / 25,000 rows
80 train / 20 validation
Record before the action

## Fresh rollouts

20 shared test cases
Distance < 0.03 m
Speed < 0.05 m/s
Throughout final 0.5 s

**Simulation success is sampled, task-specific, and model-dependent.**